

The AI Engineer Interview Playbook

v0.1 — Energiemissie Interview Edition

Mike Pattyn

Inhoudsopgave

Cover	3
Hoe dit boek te gebruiken	3
Avond vóór het gesprek (45–60 min)	3
Ochtend van (10 min)	3
Tijdens oefengesprek met recruiter	3
Wat dit boek níet is	3
Inhoudsopgave	3
Leeswijzer call-outs	4
Versie	4
Deel I — Mike Pattyn	5
Wie ben jij als engineer?	5
Carrière-arc (als verhaal)	5
Superpowers (gemapt op de vacature)	6
Positionering t.o.v. de rolstitel	7
Elevator pitches	7
Engineering filosofie (zodat je consistent klinkt)	8
Anti-patterns in jouw antwoorden	8
Jouw “anders dan gemiddelde senior”	8
Deel II — Energiemissie en de rol	10
Wat doen ze écht?	10
Vacature zin-voor-zin	12
90-dagenplan als jouw pitch	13
Waarom Energiemissie voor jou?	14
Klantreizen om over te praten (zonder hun internals te claimen)	15
Jouw “waarom nu”-verhaal (90 seconden)	15
Deel III — CEO Pieter Sleetboom	16
Wie zit er tegenover je?	16
Wat meet een CEO in dit gesprek?	16
Hoe Pieter waarschijnlijk vragen stelt	17
Mike’s Playbook: gesimuleerde CEO-loop	18
Sparring met een CTO die bouwt	19
Mentale checklist vlak voor je binnenloopt	19
Meer CEO-simulaties	19
Signalen tijdens het gesprek (lees de kamer)	20
Afsluiting die sterk voelt	20

Deel IV – Vraagbibliotheek (~35)	21
A. Persoonlijk & motivatie	21
B. Ownership & werkwijze	22
C. .NET, architectuur, fullstack	23
D. AI, LLM, agents, RAG	24
E. Data, schaal, security	26
F. Cultuur & AI-first	26
G. Gedrag & lastige vragen	27
Top 12 – als je maar één uur hebt	27
H. Whiteboard & ontwerp vragen	27
Antwoord-timing	28
Recruiter-oefening (morgen)	28
Deel V – Projecten en STAR-bibliotheek	29
Projectkaart	29
Yaya – bewijs van ownership én AI engineering	29
Echo – bewijs van AI side-project met oordeel	30
Flyingdarts – bewijs van cloud en makersmentaliteit	31
Donatieplatform – bewijs van schaal en verantwoordelijkheid	31
Hoe je projecten “verkoopt” in één adem	32
Mini-STAR cheat (8 kaarten)	32
Whiteboard: Yaya in 3 minuten	32
STAR – Client feedback loop	33
STAR – Parallel leadership (Scrum/coach)	33
Frasering: van techniek naar Energiemissie	33
Wat je zwijgt (bewust)	34
Deel VI – AI Native cheats (interview-ready)	35
Cheatsheet – definities in mensentaal	35
Agents – whiteboard in 60 seconden	35
RAG – wanneer wel, hoe goed	36
Evals – minimale productiebar	36
Tracing & observability	36
Fallbacks & guardrails (energy SaaS)	37
MCP & CLI voor het team	37
Frameworks – hoe je erover praat	38
.NET + AI – brugzin	38
Cost & latency – één minuut	38
Security one-liner	38
10 flitsvragen (antwoord ≤20 seconden)	39
Uitlegpatronen (CEO → CTO)	39
Mini-ontwerp: “Besparingssignaal”-feature	39
Begrippen die je mag durven corrigeren	40
Week-1 leerplan AI×energie (zodat je concreet klinkt)	40
Deel VII – Salaris, hybride en last-5-minutes	41

Wat de vacature al vastlegt	41
Onderhandelingsprincipes	41
Vragen die wél mogen over voorwaarden	42
Last 5 minutes — cheat sheet	42
Avond-vóór checklist	43
Ochtend-van (10 minuten)	43
Onderhandelingscripts (kort)	43
Presence (praktisch, geen zweverig advies)	44
Na het gesprek (dezelfde dag)	44

Cover

The AI Engineer Interview Playbook

v0.1 — Energiemissie Interview Edition

Persoonlijk handboek voor het gesprek met **Energiemissie|Trenton**

Rol: **AI-Native Full Stack Engineer**

CEO: **Pieter Sleeboom** · Locatie: Diemen (hybride)

Dit is geen generieke sollicitatiegids.

Dit is jouw speelboek: bewijs, antwoorden, en de 90-dagenpitch.

Hoe dit boek te gebruiken

Avond vóór het gesprek (45–60 min)

1. Deel II + III — bedrijf, vacature, CEO (eenmaal rustig)
2. Deel V — Yaya ownership + agents STAR hardop
3. Deel IV — Top 12 vragen scannen
4. Deel VII — last-5 + salarisrange invullen

Ochtend van (10 min)

Alleen Deel VII last-5 + 30-seconden pitch hardop.

Tijdens oefengesprek met recruiter

Gebruik Deel IV als vraagbank; laat haar doorvragen op V3, V11, V21, V32.

Wat dit boek niet is

Geen 250-vragen encyclopedie. Geen Big Tech-editie.

Scope: **40–60 pagina's**, scherp op vrijdag.

Inhoudsopgave

1. Deel I — Mike Pattyn
2. Deel II — Energiemissie en de rol

3. Deel III — CEO Pieter Sleeboom
4. Deel IV — Vraagbibliotheek (~35)
5. Deel V — Projecten en STAR-bibliotheek
6. Deel VI — AI Native cheats
7. Deel VII — Salaris, hybride en last-5-minutes

Bronnen: vacature Recrutee, mikepattyn.framer.website, energiemissie.nl (publiek), eigen projectkennis Yaya/Echo/Flyingdarts.

Leeswijzer call-outs

Door het boek heen zie je vier blokken:

- **CEO Insight** — wat Pieter waarschijnlijk meet
- **What they're really asking** — de onderliggende vraag
- **Perfect answer** — kerndraai van een sterk antwoord
- **Don't say this** — alarmbel-antwoorden

Gebruik ze als spiergeheugen, niet als toneelstuk. Jouw woorden, hun bewijs.

Versie

Veld	Waarde
Versie	v0.1
Editie	Energiemissie Interview Edition
Doelomvang	40–60 pagina's
Taal	Nederlands
Status	Interview-ready manuscript

Latere versies (buiten scope nu): uitbreiden naar generieke Senior AI Engineer edition.

Deel I — Mike Pattyn

Wie ben jij als engineer?

Niet je CV. Je **professionele identiteit** — zodat je in het gesprek klinkt als iemand met een kompas, niet als iemand die vacatureteksten napraten.

Kerndefinitie (leer dit)

Ik ben een product-minded full stack engineer die ownership neemt over de hele keten — van UI tot API tot AI-agents in productie. Ik werk het best in kleine teams waar tempo, kwaliteit en gebruikersfeedback samenkomen. AI is voor mij geen speeltje: het is een multiplier die ik ontwerp, evalueer en operationaliseer.

Drie ankers:

1. **Ownership** — primary engineer 0→1 (Yaya)
2. **Fullstack met backend-zwaarte** — .NET/C# + Angular + cloud
3. **AI als productdiscipline** — agents, RAG, evals, tracing — live

Hoe jij denkt

- Begin bij het gebruikersprobleem, niet bij het framework
- Snijd scope tot een vertical slice die leert
- Maak failure modes expliciet (zeker bij AI)
- Verwerf feedback vroeg (client loop op Yaya)
- Automatiseer kwaliteit (tests, evals, CI) zodat tempo houdbaar blijft

Hoe jij leert

- Bouwen > alleen consumeren (Echo, Flyingdarts)
- Nieuwe stack leren door een echt pad naar productie
- Tools zoals Cursor/Claude Code gebruiken om *sneller* te leren én te leveren — niet om na te denken te vermijden

What they're really asking wanneer ze “vertel over jezelf” zeggen:

Ben jij de persoon die de AI-laag kan dragen zonder te verdwalen in hype?

Carrière-arc (als verhaal)

Hoofdstuk A — Fundament (.NET + multi-stack)

VDAB .NET/C#, daarna jaren op een donatieplatform met **100.000+ users** en **>€1M/maand** flows. Angular, React, mobile, .NET, Azure/AWS, payment providers. Rollen naast development: Scrum Master, Product Owner.

Les die je meeneemt: software raakt geld en vertrouwen. Discipline rond integraties en stabiliteit is geen “enterprise overhead”.

Hoofdstuk B — Maker buiten werktijd

Flyingdarts: real-time product + AWS serverless. Bewijst dat je producten wilt bestaan, niet alleen tickets sluiten.

Hoofdstuk C — Lead + AI-native (nu)

Sinds 2023: lead developer op Yaya — AI-powered real-time platform. Van idee naar productie in ~12 maanden. Vier Angular-apps, ASP.NET Core CQRS, Python agents (LangGraph, LiveKit, RAG), Azure.

Les: AI in productie eist dezelfde engineering rigor als de rest van het platform — plus evals en cost control.

Hoofdstuk D — Echo (nu)

Discord-agent met intent routing en RAG. Side-project dat oordeel + agent patterns scherpt.

De boog in één zin

Van betrouwbare fullstack op schaal → naar ownership van AI-producten end-to-end → klaar om een energy-intelligence layer in een serieuze SaaS mede neer te zetten.

Superpowers (gemapt op de vacature)

1. Ownership zonder permission theater

Waarom belangrijk bij Energiemissie: autonomie is een cultuurpijler; 90-dagenplan eist dat jij trekt.

Bewijs: primary engineer Yaya; architectuur + productrichting.

Interviewregel: praat in “ik was accountable voor...”, niet “wij als team deden...”, behalve waar credit delen eerlijk is.

2. .NET/C# als ruggengraat

Waarom: expliciete must-have.

Bewijs: ASP.NET Core 9, CQRS/MediatR, EF Core, SignalR, Azure integrations, testbaarheid.

Interviewregel: één concrete design choice noemen (bijv. waarom CQRS daar paste).

3. Agents & tool use in productie

Waarom: “LLMs als product... agents en tool use... live ging”.

Bewijs: LangGraph workflows, LiveKit agents, Echo routing.

Interviewregel: definieer agent scherp; noem policies/fallbacks.

4. Evals, tracing, fallbacks

Waarom: expliciet in vacature + ISO-context.

Bewijs: DeepEval/semantic evaluation, OpenTelemetry/App Insights, E2E audio tests.

Interviewregel: “geen magie zonder metriek”.

5. AI-first execution (“jij stuurt, AI bouwt”)

Waarom: culturele litmus test van de vacature.

Bewijs: dagelijkse agentic tooling; snelle levering mét reviewdiscipline.

Interviewregel: sturen = criteria + architectuur + review; bouwen = accelerated implementation.

6. Product sense & UX-brug

Waarom: samenwerking met Product; features moeten klantwaarde raken.

Bewijs: client feedback loop; design/engineering brug op je site.

Interviewregel: koppel features aan euro’s/CO₂/tijdswinst voor multi-site klanten — niet aan modelnamen.

7. Schaal-instinct

Waarom: 250k aansluitingen; eerdere 100k users.

Bewijs: high-traffic/payment verleden; monorepo discipline; cloud patterns (Azure/AWS).

Interviewregel: praat over tenant isolation, query/AI cost, async work.

Don’t say this

“Mijn superpower is dat ik alles kan.”

Kies er drie en maak ze belachelijk concreet.

Positionering t.o.v. de rolstitel

De pagina zegt Full Stack Engineer; de header AI-Native Full Stack Engineer. Positioneer je als:

Full stack engineer met AI-native productervaring — sterk op .NET en agentic systems — die de energy intelligence layer kan mede-architecten en shippen.

Niet:

“Pure ML engineer” of “alleen Angular dev die AI leuk vindt”.

Elevator pitches

30 seconden

“Ik ben Mike — full stack, .NET-zwaar, met productie-AI. Ik bracht als primary engineer een AI-communicatieplatform van nul naar live in een jaar. Ik wil diezelfde ownership gebruiken om jullie agentic energy-intelligence features te bouwen op jullie data platform.”

90 seconden

Voeg toe: Angular + ASP.NET CQRS + LangGraph/RAG/evals; cultuurmatch AI-first; 90 dagen eerste feature + standaard neerzetten.

5 seconden (handdruk)

“AI-native fullstack met .NET en agents in productie — klaar om jullie intelligence layer te shippen.”

Engineering filosofie (zodat je consistent klinkt)

1. Outcome > activity

Tickets afvinken is geen ownership. Ownership is: de gebruiker kan iets dat gisteren niet kon — veilig, observeerbaar, houdbaar.

2. Vertical slices

Liever één smalle flow die end-to-end werkt (UI → API → data → agent → eval) dan horizontale lagen die “bijna klaar” zijn.

3. Saai waar het moet, slim waar het mag

Auth, tenant isolation, factuurpaden, migrations: saai en streng.

Planning, samenvatten, scenariodenken: hier mag AI schitteren — mét guards.

4. Feedback is een feature

Op Yaya was de client-loop geen bijzaak. Bij Energiemissie: early exposure aan energievoordelen/support-cases maakt AI-features relevanter.

5. Tools vermenigvuldigen jou — ze vervangen je oordeel niet

“Jij stuurt, AI bouwt” betekent: jij blijft de architect van kwaliteit. Als je dat niet wilt dragen, past deze cultuur niet — en jij past niet bij hen.

Anti-patterns in jouw antwoorden

Anti-pattern	Vervang door
“Wij als team...” (altijd)	“Ik was accountable voor...; het team droeg bij aan...”
Tech dump	Eén keuze + waarom + resultaat
“Ik leer snel” zonder plan	“Week 1–2 doe ik X, Y, Z”
Bescheidenheid die bewijs wist	Feiten noemen zonder opscheppen
Overclaim MCP/energie	Eerlijk + brug + hoe je het inhaalt

Jouw “anders dan gemiddelde senior”

Niet omdat je “beter” bent — omdat je **combinatie** zeldzaam is in NL-vacatures:

- Echte .NET-diepte
- Echte agentic productie (niet alleen wrappers)

- Frontend genoeg om af te maken
- 0→1 ownership
- Side-projects die oordeel tonen (Echo)
- Schaal-ervaring met money-paths

Dat is de zin die je mag verdienen in het gesprek — mits elk deel bewijsbaar blijft.

Deel II — Energiemissie en de rol

Wat doen ze écht?

Energiemissie|Trenton is geen “energie-startup met een dashboard”. Het is een gevestigde B2B SaaS-speler in professioneel energiebeheer: software die organisaties helpt om verbruik te zien, kosten te beheersen en verduurzaming te sturen — over veel locaties tegelijk.

Publieke feiten waarop je mag steunen:

- Productlijnen: **Aansluitingenregister**, **Energiemonitor**, **Factuurcontrole**
- Klanten in overheid, zorg, retail en andere multi-site organisaties
- Orde van grootte: duizenden organisaties; de vacature praat over **250.000+ energieaansluitingen**
- Koppelingen met leveranciers, netbeheerders en meetbedrijven
- ISO 9001 en ISO 27001
- Ruim vijftien jaar domeinkennis
- Fusie Energiemissie + Trenton; groei- en professionaliseringsfase onder nieuwe CEO

De kern van hun waardepropositie: *energiedata omzetten in actie* — niet alleen rapporten tonen, maar grip geven op euro's en CO₂.

CEO Insight

Pieter komt uit B2B SaaS-groei. Hij denkt in markten, productwaarde en schaalbare software — niet in “interessante AI-demo's”. Koppel AI altijd aan meetbare klantuitkomst.

Waarom AI nu?

De vacature zegt het hardop: ze staan aan de vooravond van een **AI-native product rebuild**. Dat betekent:

1. Er bestaat al een werkend SaaS-platform met echte data en echte klanten.
2. AI moet daar *in* landen — als energy intelligence layer — niet ernaast als chatbot-gadget.
3. Ze zoeken iemand die dat mede-architecteert en de eerste betaalde features ship't.

Wat ze *niet* zoeken: iemand die “ChatGPT in een sidebar” plakt. Wat ze *wél* zoeken: agentic features voor analyses, scenario's en verduurzamingspaden, geïntegreerd in het bestaande landschap, bovenop een data platform met honderdduizenden aansluitingen.

Producten in interviewtaal

Je hoeft geen sales pitch te houden. Wel drie zinnen per product:

Aansluitingenregister — “Bron van waarheid over objecten en energieaansluitingen. Zonder dit is elke AI-feature blind of gevaarlijk.”

Energiemonitor — “Meetdata, trends, benchmarks. Dit is de brandstof voor analyses en scenario's.”

Factuurcontrole — “Automatisering van controle op energiefacturen. Hier zit tijdswinst én geld; AI mag helpen routeren en uitleggen, niet stil fouten maken.”

Voor wie bouwen ze?

Segment	Pijn (interview-niveau)	AI-haak
Overheid	Veel locaties, rapportage, verduurzamingsdruk	Scenario's + uitlegbare inzichten
Zorg	Kosten drukken, complexe gebouwen	Anomalieën + prioritering
Retail multi-site	Marges, veel meters, investeringen volgen	Benchmarks + “waar eerst ingrijpen?”

Krachtenveld (publiek, voorzichtig)

Zeg niet dat je concurrenten tot in detail kent. Wél:

- Energiebeheer-software is een volwassen markt; onderscheid zit in data-diepte, integraties, sectorfit en nu **AI die echte actie aanzet**
- Fusie Energiemissie|Trenton + nieuwe CEO = groeifase en professionalisering
- ISO-certificering = trust voor overheid/zorg; AI moet dat versterken

Don't say this

“Jullie zijn behind op AI.”

Zeg: “Jullie hebben het zeldzame voordeel van data + klanten; de rebuild is het moment om intelligence native te maken.”

Wat “energy intelligence” wél en niet is

Wel	Niet
Agent die tools gebruikt op jullie data	Vrije chat zonder bronnen
Scenario's met aannames en ranges	Eén magisch “bespaar 30%”-getal
Workflows in de bestaande SaaS	Losse AI-microsite
Evals + fallback	“Het model is slim genoeg”
Betaalde feature met packaging	Eindeloze interne PoC

Vacature zin-voor-zin

“Bouw de agentic AI-features in onze SaaS-app bovenop 250.000 energieaansluitingen”

What they’re really asking

Kun jij AI bouwen op schaalbare, gevoelige, multi-tenant data — niet op een speeltuin-dataset?

Jouw haak: Yaya draaide al agents + RAG + real-time pipelines in productie. Eerder werkte je op software met 100.000+ users. Je denkt in data contracts, observability en failure modes — niet alleen prompts.

“Geen chatbot maar agentic features volledig geïntegreerd”

Perfect answer

“Ik bouw liever een agent die een verduurzamingsscenario kan doorrekenen met tools op jullie data, dan een chat die tekst genereert. De waarde zit in tool use, guardrails en UI die in de bestaande SaaS-flow past.”

“Je werkt fullstack, van interface tot data platform”

Ze willen geen pure ML-onderzoeker en geen pure UI-builder. Ze willen iemand die:

- .NET/C# backend serieus neemt
- frontend genoeg beheerst om features af te maken
- data platform layer begrijpt als voeding voor AI

Jouw haak: ASP.NET Core CQRS + Angular monorepo + Python AI-services — één ownership-lijn.

“MCP- en CLI-tooling bouwen voor het team en onze agents”

Dit is een **nice-to-have die ze bijna als must behandelen**. MCP = Model Context Protocol: gestandaardiseerde tools die agents (en developers) kunnen aanroepen.

Don’t say this

“MCP ken ik niet, maar ik leer snel alles.”

Zeg liever: “Ik heb agents met tool calling en developer tooling gebouwd; MCP is de standaardisering daarvan — dat zet ik in de eerste weken structureel neer.”

“In het verleden schreef je zelf code — dat heb je inmiddels ver achter je gelaten. Jij stuurt, AI bouwt.”

Dit is de culturele litmus test.

What they're really asking

Gebruik jij AI als multiplier, of als crutch? Kun je nog steeds architectuur, reviews en kwaliteit dragen terwijl agents de bulk typen?

Jouw framing:

- Jij definieert probleem, constraints, acceptance criteria, evals
- Agents/implementatie-tools genereren en itereren snel
- Jij blijft accountable voor productgedrag, security, kosten

Niet: “ik typ niets meer.” Wel: “ik stuur systemen en mensen+AI naar outcome.”

Cultuur: Autonomie · AI-first · Innovatie

Pijler	Wat het betekent in het gesprek	Hoe jij het bewijst
Autonomie	Geen ticket-wachter	Yaya: primary engineer, architectuur + productrichting
AI-first	AI is default lens, niet add-on	Live agents, RAG, evals; dagelijks Claude Code/Cursor
Innovatie	Ship, meet, bijstellen	0→1 in ~12 maanden met client feedback loop

Aanbod lezen als signaal

- Hybride Diemen (1 of 2 thuisdagen afhankelijk van uren) → ze willen fysieke nabijheid in een klein team
- Budget voor tools/AI-credits/conferenties → ze menen AI-first
- “CTO die zelf bouwt” → technische diepgang wordt gewaardeerd; geen pure people-manager boven je
- Directe invloed op productstrategie → ze verwachten mening, niet alleen uitvoering

90-dagenplan als jouw pitch

Gebruik dit als concreet verhaal — niet als belofte dat je hun roadmap kent.

Week 1–2 — Onderdorpelen

Doel: sneller dan gemiddeld nuttig worden in het energiedomein + platform.

Acties die je hardop mag noemen:

1. Meelopen met Product/Support: welke vragen stellen gemeenten en zorginstellingen écht?
2. Data model van aansluitingen/meters/facturen snappen op het niveau “wat mag een agent lezen/schrijven?”
3. Bestaande engineering practices, auth, environments, observability inventariseren
4. Met CTO: waar zit de AI-native rebuild-grens — brownfield integratie vs. groene velden?

Deliverable van jou: een korte “AI opportunity map” — 5 feature-kansen gerankt op klantwaarde × data-readiness × risico.

Maand 1 — Eerste AI-feature

Doel: iets lives dat klantwaarde levert — betaald pad, geen interne demo.

Voorbeelden van *richting* (niet claimen dat dit hun backlog is):

- Anomalie-/besparingssignaal met uitleg + bronverwijzing naar meetdata
- Scenario-assistent: “wat als we deze set gebouwen isoleren / laden verschuiven?”
- Agent die factuurcontrole-uitzonderingen samenvat en naar de juiste workflow routeert

Jouw engineering standaard bij ship:

- Tool use op echte platform-API's / data access layer
- Tracing + fallback (graceful degrade naar klassieke UI/rules)
- Minimale eval-set vóór productie
- Kostenplafond per request/tenant

Maand 2-3 — De AI-laag

Doel: mede-eigenaar van hoe Energiemissie AI-native bouwt.

- Herbruikbare agent/runtime-patronen (.NET + eventueel Python/services)
- MCP/CLI zodat het team dezelfde tools heeft als de agents
- Eval-harness + dashboards (kwaliteit, latency, cost)
- Documentatie: “zo bouwen wij AI-features hier”

Succesdefinitie uit de vacature (herformuleer in “ik”-taal):

Na 90 dagen draait er AI in productie die van mij is en directe waarde levert. Collega's komen bij mij voor hoe-vragen over agents en LLMs. Ik weet precies waar mijn werk het verschil maakt voor overheden en zorginstellingen met honderden locaties.

Waarom Energiemissie voor jou?

Drie zinnen die kloppen met jouw profiel:

1. **Impact op schaal:** software die al in het veld staat bij duizenden organisaties — AI hier is geen speeltuin.
2. **AI-native rebuild-moment:** zeldzaam om als engineer de intelligence layer mede te mogen zetten i.p.v. alleen features te plakken.
3. **Cultuurmatch:** klein team, CTO die bouwt, autonomie, AI-first zonder corporate rem — dicht bij hoe jij Yaya en Echo al werkt.

Don't say this

“Ik wil vooral remote en AI experimenteren.”

Of: “Energie klinkt wel leuk.”

Maak het specifiek: multi-site klanten, data-at-scale, paid AI features, ownership.

Klantreizen om over te praten (zonder hun internals te claimen)

Gebruik dit als *hypotheses* in gesprek met Product/CEO — toon product sense.

Reis A — Energiecoördinator gemeente

1. Ziet piekverbruik op een cluster schoolgebouwen
2. Wil weten: anomalie of seizoen?
3. Wil opties: gedrag, installatie, investering
4. Moet intern uitleggen met bronnen

AI-feature-hypothese: uitlegbare anomalie + “volgende beste vragen” + link naar meterreeksen.

Reis B — Controller zorginstelling

1. Factuurcontrole markeert afwijkingen
2. Menselijke tijd gaat naar sorteren en duiden
3. Wil sneller: kritiek vs. ruis

AI-feature-hypothese: triage-agent die uitzonderingen cluster, samenvat en naar workflow routeert — met harde guards op financiële claims.

Reis C — Retail operations

1. Vergelijkt locaties
2. Zoekt waar investering het meest oplevert
3. Monitor’t of maatregelen werken

AI-feature-hypothese: scenario-assistent + nagelmeter op “belofte vs. realisatie”.

Jouw “waarom nu”-verhaal (90 seconden)

Energiemissie heeft wat de meeste AI-teams missen: gevestigde SaaS, sectorvertrouwen en data op schaal. De vacature maakt duidelijk dat jullie geen chatbot willen, maar agentic intelligence in het product. Dat is precies mijn snijvlak: ik heb .NET-backends en Angular-frontends gebouwd, én agents met tool use, RAG en evals in productie gebracht. In de eerste negentig dagen wil ik niet “AI verkennen” — ik wil één feature live die klantwaarde raakt en tegelijk het patroon zetten hoe we hier AI-native bouwen.

Deel III — CEO Pieter Sleeboom

Wie zit er tegenover je?

Pieter Sleeboom is sinds 1 september 2024 CEO van Energiemissie|Trenton. Hij is geen “energie-nerd die toevallig CEO werd”. Hij is een **B2B SaaS-groei-executive** die het stokje overnam na een fase van groei en de fusie met Trenton.

Publiek bekende achtergrond:

- 15+ jaar ervaring als executive bij innovatieve B2B SaaS en scale-ups
- Ex-COO bij **Shypple** (digitale freight forwarder) — supply chain inzichtelijk en efficiënter maken voor grote bedrijven + organisatie laten groeien
- Bestuursrol bij AdfluenceHub (B2B platform)
- LinkedIn-positionering: energy transition SaaS, serial founder / exits — gebruik dat als context, niet als gossip

Zijn publieke boodschap bij aantreden draait om:

- duurzaamheid en energiebeheer worden belangrijker voor overheid en bedrijven
- administratie versimpelen naast verbruik optimaliseren
- oplossingen moeten meebewegen met snel veranderende klantbehoeften
- groeien **én** impact op efficiënt energiegebruik

CEO Insight

Een CEO met SaaS-achtergrond luistert naar: klariteit, ownership, klantwaarde, tempo met kwaliteit, en of jij risico's snapt (security, data, cost). Techniek moet vertaalbaar zijn naar business outcome.

Wat meet een CEO in dit gesprek?

Pieter hoeft niet elke LangGraph-node te begrijpen. Hij moet wel voelen:

Signaal	Sterk	Zwak
Ownership	“Dit heb ik end-to-end gedragen”	“Ik werkte in een team aan tickets”
AI-leverage	“Ik stuur agents + evals; zo schaal ik output”	“Ik gebruik Copilot voor autocomplete”
Product sense	“Eerste feature zou X zijn omdat klant Y...”	“Ik bouw wat Product vraagt”
Schaal	Praat over data, multi-tenant, failure modes	Alleen happy-path demo's
Cultuurfit	Autonomie + AI-first + bijstellen	Process-zwaar of “zeg maar wat ik moet doen”
Communicatie	Helder, kort, met bewijs	Jargon-muur of vage claims

Wat wil hij horen?

1. **Je snapt het bedrijf** — multi-site energiebeheer, niet “groene tech is cool”.
2. **Je snapt de rol** — agentic features in SaaS, geen chatbot-theater.
3. **Je hebt het al gedaan** — agents/tool use/.NET/fullstack in productie.
4. **Je kunt de 90 dagen invullen** — zonder hun interne roadmap te claimen.
5. **Je maakt anderen sneller** — MCP/CLI, standaarden, “collega’s komen voor hoe-vragen”.

Welke antwoorden laten alarmbellen afgaan?

Don’t say this

- “Ik ben vooral een frontend-dev die AI leuk vindt.”
- “Ik wil remote-first en maximale vrijheid.” (hybride Diemen staat expliciet)
- “Energie ken ik nog niet, maar AI is universeel.” (zonder leerplan)
- “Ik schrijf zelf geen code meer — AI doet alles.” (klinkt als abdication)
- “Chatbots zijn de toekomst van jullie portal.”
- Salaris als openingszet zonder fit te tonen

Hoe Pieter waarschijnlijk vragen stelt

CEO-vragen klinken vaak persoonlijk of strategisch, maar meten ownership en oordeel.

“Waarom moeten wij jou aannemen?”

What they’re really asking

Wat is jouw unieke combinatie voor *deze* fase van *dit* bedrijf?

Structuur:

1. Bewijs dat je 0→1 AI+product hebt gedragen (Yaya)
2. .NET + fullstack diepte voor hun stack
3. Productie-AI discipline (evals/tracing)
4. Cultuur: jij stuurt, AI bouwt — en je wilt de AI-laag hier neerzetten

“Wat zou je in de eerste 90 dagen doen?”

Hij test of je realistisch bent zonder passief te worden. Gebruik Deel II. Eindig met een meetbaar succes: “één AI-feature in productie met klantwaarde + basis voor herhaalbaar AI-engineeren.”

“Hoe werk je met AI-tools?”**Perfect answer**

“Ik behandel agents als junior engineers met supersnel typen: ik geef context, constraints en acceptance tests. Ik review diffs, dwing tracing/evals af, en hou kosten en security in de gaten. Op Yaya betekende dat LangGraph-workflows en voice agents die echt live gingen — niet alleen een prompt in een notebook.”

“Vertel over een moment waarop iets misging”

CEO’s willen leren zien + verantwoordelijkheid. Kies een Yaya-voorbeeld: AI-gedrag, real-time, of migratie — Situation → wat jij deed → wat je structureel verbeterde (tests, evals, observability).

Mike’s Playbook: gesimuleerde CEO-loop

Gebruik dit patroon bij elke zware vraag:

Pieter vraagt X

- Wat meet hij écht?
- Welk bewijs van Mike past?
- Antwoord in 60–90 seconden
- Eén concrete detail (stack/impact)
- Stop. Laat hem doorvragen.

Voorbeeld: “Waarom Energiemissie?”

Wat meet hij: motivatie die langer meegaat dan “leuke vacature”.

Mike-antwoord (kern):

Ik wil AI bouwen waar data en klanten al bestaan — niet alleen greenfield speeltuinen. Jullie zitten op honderdduizenden aansluitingen voor overheid en zorg. De AI-native rebuild is precies het moment waarop een engineer met mijn profiel — .NET, agents in productie, ownership — het verschil kan maken. En jullie cultuur van autonomie en AI-first matcht hoe ik al werk.

Waarschijnlijke doorvraag: “Oké, maar wat bouw je dan als eerste?”

→ Terug naar 90-dagen feature-hypotheses + eval/fallback discipline.

Sparring met een CTO die bouwt

De vacature noemt expliciet een CTO die zelf codeert. Implicatie voor het CEO-gesprek:

- Te vaag technisch = Pieter (of later de CTO) prikt erdoorheen
- Te diep zonder businessbrug = “slimme specialist, geen multiplier”
- Ideaal: **één laag dieper dan de vraag**, dan vertalen naar klant/risico/tempo

Als Pieter zegt “we willen agentic features”, antwoord niet alleen “LangGraph”. Zeg:

Agent = LLM + tools + state + policies. Op jullie schaal zou ik eerst de tool-laag op het data platform hard maken — wat mag een agent lezen per tenant — en dan één vertical feature shippen met evals. Framework is secundair; contracten en observatie eerst.

Mentale checklist vlak voor je binnenloopt

- ☐ Ik kan Energiemissie in 30 seconden uitleggen zonder website na te praten
 - ☐ Ik kan de rol in één zin: “agentic energy intelligence in de SaaS, geen chatbot”
 - ☐ Ik heb 3 bewijzen paraat: Yaya AI, .NET ownership, schaal/productie
 - ☐ Ik heb een 90-dagenverhaal
 - ☐ Ik ken de drie cultuurpijlers
 - ☐ Ik weet wie Pieter is (SaaS CEO, niet alleen “de baas”)
 - ☐ Ik praat over “jij stuurt, AI bouwt” zonder arrogant of lui te klinken
-

Meer CEO-simulaties

“Hoe meet je succes van AI?”

What they’re really asking

Ben jij metrics-gedreven of demo-gedreven?

Mike-antwoord:

“Op drie lagen. Product: gebruiken klanten de feature en levert het tijd/geld/CO₂-inzicht op? Kwaliteit: eval-score, escalation/fallback-rate, complaint-rate. Economie: kosten per succesvolle actie. Als alleen ‘het model antwoordt’ groen is, hebben we de verkeerde metric.”

“Wat als Product een chatbot wil?”

Mike-antwoord:

“Dan vraag ik welk job-to-be-done de chat oplost. Vaak is het echte probleem triage, scenario’s

of uitleg — dat kan een guided agent-flow zijn zonder ‘ChatGPT-sidebar’. Ik push terug met een snellere path-to-value en lagere risico’s. Als chat toch de schil wordt, dan als UI over tools met grounding — niet als vrij model.”

“Hoe overtuig je sceptische domeinexperts?”

Mike-antwoord:

“Niet met modelbenchmarks. Met shadow mode: AI stelt voor, expert beslist, we meten agreement. Daarna steeds meer autonomie waar evals het toelaten. Domeinexperts moeten de tool-laag vertrouwen — authz, bronnen, audit.”

“Waarom jij en niet een pure data scientist?”

Mike-antwoord:

“Omdat deze rol SaaS-engineering is: .NET, integratie, UI, multi-tenant security, shippen. Ik kan model-orkestratie, maar ik optimaliseer voor productoutcome in jullie stack — niet voor papers.”

Signalen tijdens het gesprek (lees de kamer)

Signaal van Pieter	Wat je doet
Hij onderbreekt met “maar wat is de business value?”	Stop tech; geef euro/tijd/risico
Hij vraagt door op ownership	Geef “ik was accountable voor X” + resultaat
Hij noemt klanten/overheid	Koppel ISO, uitlegbaarheid, geen hallucinatie-risico
Hij glimlacht om AI-tools	Toon discipline (review, evals), geen speelsheid alleen
Hij is stil	Eindig je antwoord; vul stilte niet met waffle

Afsluiting die sterk voelt

“Ik wil deze rol omdat ik hier meetbare AI kan shippen op echte energiedata — met ownership. Als jullie twijfelen over één ding, laat het mijn energiedomein-diepte zijn: die haal ik bewust in. Waar ik niet aan twijfel is of ik agents, .NET en productiekwaliteit kan dragen — dat doe ik al.”

Deel IV — Vraagbibliotheek (~35)

Formaat per vraag:

1. **Waarom** stellen ze dit?
 2. **Wat zoeken ze?**
 3. **Mike-antwoord** (kern, spreektaal)
 4. **Don't say**
 5. **Doorvraag** om op voorbereid te zijn
-

A. Persoonlijk & motivatie

V1. Vertel eens iets over jezelf

Waarom: Ijsbreker; ze horen wat jij zelf belangrijk vindt.

Wat zoeken ze? Relevantie, focus, geen levensverhaal.

Mike-antwoord:

“Ik ben full stack engineer met ongeveer negen jaar ervaring, nu lead op een AI-communicatie-platform dat ik als primary engineer van nul naar productie heb gebracht. Wat mij typeert: ownership over de hele keten — Angular, .NET, Azure en Python-agents — en productdenken dicht op de gebruiker. Ik zoek een volgende stap waar AI geen side-project is maar de intelligence layer van een SaaS met echte schaal — precies wat jullie met de AI-native rebuild doen.”

Don't say: Chronologisch CV vanaf 2010.

Doorvraag: “Wat was jouw rol precies ten opzichte van het team?”

V2. Waarom wil je bij Energiemissie werken?

Waarom: Motivatie-test.

Wat zoeken ze? Specifiek bedrijf, niet “ik zoek een baan”.

Mike-antwoord:

“Drie dingen. Eén: jullie zitten al op energiedata voor overheid en zorg — AI hier raakt euro's en CO₂, niet alleen demo's. Twee: de agentic energy-intelligence layer op 250.000+ aansluitingen is precies het soort probleem waar mijn Yaya-ervaring met agents, RAG en productie-evals op aansluit. Drie: autonomie en AI-first zonder corporate rem — dat is hoe ik al werk.”

Don't say: “Ik vind duurzaamheid belangrijk” zonder productbrug.

Doorvraag: “Wat weet je over onze producten?”

V3. Waarom moet ik jou aannemen?

Waarom: CEO-compressietest.

Wat zoeken ze? Unieke combinatie + bewijs.

Mike-antwoord:

“Omdat jullie iemand nodig hebben die .NET-SaaS wél serieus neemt én agents in productie heeft gebracht. Ik heb laten zien dat ik 0→1 kan dragen, AI kan evalueren en monitoren, en features afmaak over de hele stack. In negentig dagen wil ik een eerste AI-feature live hebben en de standaard zetten hoe jullie AI-native bouwen.”

Don't say: "Ik ben een hard worker en teamplayer."

Doorvraag: "Waar ben je minder sterk?"

V4. Waar wil je over drie jaar staan?

Waarom: Ambitie vs. flight risk.

Wat zoeken ze? Groei die hen helpt.

Mike-antwoord:

"Ik wil de engineer zijn waar het team naartoe komt voor AI-systemen in productie — architectuur, evals, tool-ecosysteem — en intussen diep genoeg in jullie energiedomein zitten om zelf kansen te zien. Niet per se een klassieke manager; wel technical leadership met productimpact."

Don't say: "Ik wil CTO worden / eigen bedrijf." (tenzij gevraagd en genuanceerd)

Doorvraag: "Hoe ga je het energiedomein leren?"

V5. Wat motiveert je het meest?

Mike-antwoord:

"Brug tussen product en engineering: iets bouwen dat gebruikers echt voelen, snel itereren op feedback, en techniek hoog houden. AI is voor mij een multiplier op dat pad — niet het doel op zich."

Don't say: Alleen "nieuwe tech".

B. Ownership & werkwijze

V6. Hoe ziet ownership eruit in jouw werk?

Mike-antwoord:

"Op Yaya betekende ownership: ik was accountable voor architectuurkeuzes, oplevering en kwaliteit over frontend, API en AI-services. Ik wachtte niet tot iemand de AI-stukjes 'oppakte' — ik maakte ze onderdeel van het productpad, inclusief tests en telemetry."

Don't say: "Ik pak tickets van de board."

V7. Hoe ga je om met onduidelijke requirements?

Mike-antwoord:

"Ik maak de kleinste scherpe hypothese: welk gebruikersprobleem, welk succescriterium, welke data hebben we nodig. Dan korte spike of vertical slice, feedback van stakeholder, bijstellen. Op Yaya was de client-loop continuous build-measure-learn."

Don't say: "Dan wacht ik tot Product het uitschrijft."

V8. Vertel over een conflict of meningsverschil

Structuur: Situation → jouw belang → hoe je het oploste → resultaat.

Mike-antwoord (template):

Kies een technisch/product conflict (scope vs. kwaliteit, of framework-keuze). Toon dat je data/gebruikersrisico gebruikte, niet ego. Eindig met wat je anders deed in het proces (ADRs, spike, eval).

Don't say: Collega's zwartmaken.

V9. Wanneer heb je iets te vroeg of te laat geshipped?

Mike-antwoord:

Wees eerlijk: noem een moment waarop je scope sneed om te leren, of juist een kwaliteitshek (tests/evals) afdwong vóór release. Koppel aan AI: “bij agents is ‘werkt op mijn machine’ gevaarlijk — ik ship liever met fallback en eval-set.”

V10. Hoe werk je in een klein team naast een CTO die bouwt?

Mike-antwoord:

“Ik kom met voorstellen en trade-offs, niet alleen problemen. Ik spar op code/architectuur-niveau, maar ik sleep niet alles naar de CTO — ik maak zoveel mogelijk zelf af en escaller bij echte platformkeuzes.”

Don’t say: “Ik heb veel mentoring nodig.”

C. .NET, architectuur, fullstack

V11. Hoe diep is jouw .NET-ervaring?

Mike-antwoord:

“Recent: ASP.NET Core 9 API greenfield met CQRS/MediatR, pipeline behaviors, EF Core 9 met tientallen migrations, SignalR hubs, JWT-flows, FluentValidation, integratie met Azure-services, xUnit met echte SQL-testdatabase. Dat is mijn primaire backend-spier. Daarvóór jaren .NET in productie op grotere user bases.”

Don’t say: “Ik doe vooral frontend, .NET ken ik een beetje.”

V12. CQRS — waarom wel/niet?

Mike-antwoord:

“Op Yaya paste CQRS omdat we duidelijke commands/queries, validatie en behaviors wilden zonder fat controllers. Het is geen religie: bij eenvoudige CRUD zou ik het niet forceren. Op een AI-laag zou ik commands voor tool-side-effects en queries voor retrieval/analytics strikt scheiden.”

Don’t say: “CQRS is always best practice.”

V13. Hoe ontwerp je APIs voor AI-tool use?

Mike-antwoord:

“Tools moeten smal, idempotent waar kan, goed geautoriseerd per tenant, en traceerbaar zijn. Liever tien scherpe tools dan één god-endpoint. Contract-first: schema’s, errors, timeouts. De LLM is onbetrouwbaar; de tool-laag moet saai en hard zijn.”

Doorvraag: “Hoe doe je authz per aansluiting/organisatie?”

V14. Frontend — hoe sterk ben je?

Mike-antwoord:

“Sterk genoeg om features end-to-end te leveren. Op Yaya vier Angular-apps in een monorepo, shared libraries, NgRx + SignalR, Playwright/Storybook. Ik ben backend-zwaarder, maar ik lever geen ‘API over de schutting’.”

Don’t say: “Frontend boeit me niet.”

V15. Hoe denk je over microservices vs. modular monolith?

Mike-antwoord:

“Start coherent; split op team/scale/failure-boundaries. AI-services (Python) naast .NET kan logisch zijn vanwege ecosystem — maar data contracts en observability moeten eerst kloppen. Voor een rebuild: modulariteit > microservices-theater.”

D. AI, LLM, agents, RAG

V16. Wat is een agent — in jouw woorden?

Mike-antwoord:

“Een systeem waarbij een model niet alleen tekst geeft, maar een doel nastreeft via tools, state en policies — met menselijke of automatische stops. Op Yaya: LangGraph-workflows met tools en streaming; op Echo: routing naar RAG of search.”

Don’t say: “Alles met een LLM is een agent.”

V17. Wanneer géén agent gebruiken?

Mike-antwoord:

“Als een deterministische pipeline of ruleset beter, goedkoper en uitlegbaarder is. Agents waar oordeel + tool-keuze nodig is; klassieke software waar het pad vastligt. Op energiedata zou ik compliance-gevoelige writes nooit ‘vrij’ aan een agent geven zonder harde guards.”

V18. Hoe heb je tool use gebouwd?

Mike-antwoord:

“Tools als expliciete functies met schema’s; het model kiest/vult arguments; runtime valideert en executeert; resultaten terug in de loop. State/checkpointing via LangGraph zodat lange flows hervatbaar zijn. Logging van tool calls is non-negotiable.”

V19. RAG — hoe leg je het uit aan een niet-tech CEO?

Mike-antwoord:

“Eerst zoeken we in jullie eigen kennis en data naar relevante stukken, dan laat ik het model daarop antwoorden met bronnen. Zo verklein je hallucinaties en kun je aantonen wáárom iets gezegd wordt — cruciaal bij energieadvies.”

Don’t say: Alleen “vector database”.

V20. Hoe ga je om met hallucinaties?

Mike-antwoord:

“Preventie: retrieval, grounded prompts, tool results als source of truth. Detectie: evals, human review op riskante flows. Mitigatie: weigeren/escaleren, fallback UI, duidelijke onzekerheid. Nooit stille verzinsels in factuur- of compliance-paden.”

V21. Evals — wat heb je concreet gedaan?

Mike-antwoord:

“Semantic evaluation en DeepEval-achtige flows op Yaya: checklists/embedding-matching, tracing, plus E2E met generated audio om AI-gedrag te valideren. Mijn standaard: geen productie

zonder minimale testset en regressie op kritieke intents.”

Doorvraag: “Hoe groot was je testset?” — Wees eerlijk over orde van grootte en dat je die hier wilt professionaliseren.

V22. Tracing en monitoring van AI in productie

Mike-antwoord:

“OpenTelemetry/App Insights op het platform; voor AI: log prompts/tool calls/latency/token cost/error rates (met privacy-redaction). Alerts op drift: opeens meer fallbacks of duurdere traces. Zonder tracing debugg je magie.”

V23. Fallback-logica

Mike-antwoord:

“Als retrieval zwak is of het model faalt: degradeer naar bewezen UI/rules, toon ‘kan dit niet betrouwbaar’, of queue voor menselijke review. Liever een saai correct pad dan een charmante foute AI.”

V24. Kostencontrole

Mike-antwoord:

“Budgets per feature/tenant, caching van retrieval, kleinere modellen voor routing, grotere alleen waar nodig, batch waar kan, max tokens, en productkeuzes: niet alles hoeft realtime LLM.”

CEO Insight: Kosten = productfeature, geen nasleep.

V25. MCP — ken je dat?

Mike-antwoord:

“MCP is een standaard om tools/context aan agents en devtools te hangen. Ik heb agents met tool calling gebouwd; MCP is de portable laag daaroverheen. In de eerste weken zou ik team-CLI/MCP-servers willen die dezelfde veilige data-tools exposen als productie-agents — zodat ‘jij stuurt, AI bouwt’ ook voor het engineering team geldt.”

Don’t say: “Nooit van gehoord.” (zonder direct brug te slaan)

V26. Claude Code / Copilot — hoe werk je dagelijks?

Mike-antwoord:

“Ik geef agents repo-context, acceptance criteria en constraints; zij genereren; ik review architectuur, security en tests. Ik gebruik ze om sneller te verkennen en te implementeren, niet om verantwoordelijkheid te outsourcen. Past exact bij jullie ‘jij stuurt, AI bouwt’.”

Don’t say: “Ik plak wat Copilot geeft.”

V27. Hoe zou een eerste energy-intelligence feature eruitzien?

Mike-antwoord:

“Ik zou met Product één vertical kiezen met duidelijke euro/CO₂-waarde — bijvoorbeeld uitlegbare besparingssignalen of scenario’s op een subset aansluitingen. Technisch: read-only tools op het data platform, grounded answers, eval-set met echte cases, fallback naar bestaande monitor-UI. Geen chatbot als primaire UX.”

Don’t say: Concrete interne features claimen alsof je hun backlog kent.

E. Data, schaal, security

V28. 250.000 aansluitingen — waar let je op?

Mike-antwoord:

“Tenant isolatie, query-kosten, indexing, async jobs voor zware analyses, rate limits op AI, en dat agents nooit ‘alles ophalen’ als default hebben. AI maakt slechte data access patterns duurder en gevaarlijker.”

V29. Security / ISO — hoe raakt dat AI?

Mike-antwoord:

“Jullie zijn ISO 27001 — AI mag dat niet ondermijnen. Geen secrets in prompts, strikte RBAC op tools, audit logs, dataminimalisatie, en leverancierskeuzes voor models met duidelijk data policy. Ik behandel tool calls als privileged API calls.”

Don’t say: “Security regelt iemand anders.”

V30. Multi-tenant fout: agent lekt data tussen organisaties

Mike-antwoord:

“Dat is severe. Design: authz in de tool-laag, nooit alleen in de prompt. Tests voor cross-tenant. Bij incident: kill switch, audit, klantcommunicatie met Product/CTO. Preventie > sorry.”

F. Cultuur & AI-first

V31. Wat betekent AI-first voor jou?

Mike-antwoord:

“Voordat we iets bouwen of automatiseren, vraag ik: welk deel kan een model+tools beter, sneller of goedkoper — en welk deel moet deterministisch blijven? AI-first is een ontwerpreflex, geen hype-checkbox.”

V32. “Jij stuurt, AI bouwt” — ben je het daarmee eens?

Mike-antwoord:

“Ja, als sturen betekent: probleemkadering, architectuur, kwaliteitspoorten, productie-accountability. Nee, als het betekent: blind genereren zonder te snappen wat er live gaat. Ik wil juist sneller shippen *omdat* ik strenger stuur.”

V33. Hoe leer je een nieuw domein (energie) snel?

Mike-antwoord:

“Shadow support/product, glossary bouwen, één klantjourney end-to-end tekenen, en meteen een kleine feature gebruiken als leervoertuig. Domeinkennis plakt beter aan concrete shipping dan aan alleen documentatie.”

G. Gedrag & lastige vragen

V34. Wat is je grootste zwakte voor deze rol?

Mike-antwoord:

“Ik heb geen jaren energiedomein-diepte — dat ga ik bewust inhalen in week 1–2. Technisch compenseer ik met ervaring in data-heavy SaaS en productie-AI. Waar ik scherp op blijf: niet te snel ‘slim’ AI’en vóór de data contracts kloppen.”

Don’t say: “Ik werk te hard” / “Ik ben perfectionist.”

V35. Salarisverwachting?

Zie Deel VII. Kort: range + dat fit en scope eerst komen; hybride/Diemen en AI-budget meewegen.

Don’t say: Eerste zin van het gesprek.

V36. Heb je nog vragen aan ons? (stel er 3)

1. “Hoe ziet succes van de AI-native rebuild eruit over zes maanden — productmetrics, niet alleen tech?”
2. “Wat is vandaag de grootste frictie tussen data platform en feature teams?”
3. “Hoe beslissen Product en engineering welke AI-features betaald/packaged worden?”

Bonus aan Pieter: “Waar wil jij dat deze hire jou persoonlijk ontzorgt in het eerste halfjaar?”

Top 12 — als je maar één uur hebt

1. V1 Vertel over jezelf
 2. V2 Waarom Energiemissie
 3. V3 Waarom jij
 4. V6 Ownership
 5. V11 .NET diepte
 6. V16 Wat is een agent
 7. V18 Tool use
 8. V21 Evals
 9. V25 MCP
 10. V26 Claude Code-workflow
 11. V27 Eerste feature
 12. V32 Jij stuurt, AI bouwt
-

H. Whiteboard & ontwerp vragen

V37. Schets hoe je een agent op ons data platform zou zetten

Waarom: Architectuurhoofd.

Mike-antwoord (spreek terwijl je tekent):

“UI in bestaande SaaS → orchestration service → LLM → tool layer → tenant-scoped data APIs. Naast de flow: tracing, eval harness, kill switch. Writes alleen via allowlisted tools. Ik begin read-

heavy.”

Don’t say: Alleen een cloud-vendor logo-tekening zonder authz.

V38. Hoe test je non-deterministische systemen?

Mike-antwoord:

“Splits deterministische tool-logic (gewone tests) van modelgedrag (evals met toleranties, snapshot van tool-keuzes, human review op riskante classes). Flaky tests aanpakken met vastgelegde fixtures en seed waar mogelijk.”

V39. Wat is het verschil tussen LangGraph en ‘gewoon’ een for-loop met prompts?

Mike-antwoord:

“Een graph/state machine maakt branches, retries, human-in-the-loop en checkpointing expliciet. Een for-loop kan werken tot je productiecomplexiteit krijgt — toen heb ik op Yaya juist state/tool orchestration nodig gehad.”

V40. Hoe voorkom je dat AI-features een second system worden?

Mike-antwoord:

“Zelfde auth,zelfde design system,zelfde release-train,zelfde observatie. AI is een capability inside the product — geen schaduw-IT. MCP/CLI helpt het team dezelfde capabilities te delen.”

Antwoord-timing

Type vraag	Doeltijd
Vertel over jezelf	60–90s
Waarom wij / waarom jij	45–60s
STAR	75–120s
Tech deep-dive	90s + “ik kan dieper”
Mening/strategie	45s, dan stoppen

Als je merkt dat je >2 minuten praat zonder adem: afronden met resultaat + brug naar Energiesmissie.

Recruiter-oefening (morgen)

Draai minimaal deze vijf hardop, met timer:

1. V1 + V2 aan elkaar (2 min totaal)
2. V3 Waarom jij
3. V11 .NET
4. V21 Evals
5. V32 Jij stuurt, AI bouwt

Laat de recruiter expres doorvragen: “Waarom niet chatbot?” en “Ken je MCP?”

Deel V — Projecten en STAR-bibliotheek

Herschrijf projecten als **bewijs**, niet als CV-opsomming. In het gesprek noem je maximaal één project per antwoord, met één scherpe les.

Projectkaart

Project	Bewijs-type	Wanneer inzetten
Yaya	Ownership + AI engineering + fullstack .NET/Angular	Primaire story voor bijna alles
Echo	Side-project AI agent + RAG + product judgment	Nice-to-have, MCP/agents, “bouw je zelf ook?”
Flyingdarts	Cloud/serverless + 0→1 productdenken	AWS, schaalbaarheid, eigenaarschap buiten werk
Donatieplatform (2018–2023)	Schaal, betrouwbaarheid, payments, multi-stack	“100k users”, verantwoordelijkheid, .NET historie

Yaya — bewijs van ownership én AI engineering

Eén zin: Ik bouwde als primary engineer een AI-powered real-time communicatieplatform van idee naar productie in ongeveer twaalf maanden.

Architectuur (interview-niveau)

Angular (4 apps, monorepo)

‡ SignalR / HTTP

ASP.NET Core 9 API (CQRS/MediatR, EF Core, SQL Server)

‡

Python AI services (LiveKit agents, LangGraph, RAG)

‡

Azure (Blob, Key Vault, App Config, Notification Hubs, App Insights)

Wat jij deed dat telt voor Energiemissie:

- Einde-tot-einde ownership in startupfase (architectuur + productrichting)
- Agents met tool/workflow-orkestratie (LangGraph) live
- Voice pipelines (STT/TTS/VAD) — bewijst real-time + multimodal thinking
- RAG met embeddings + bronnen (LlamaIndex/ChromaDB)
- Kwaliteit: DeepEval / semantic evaluation, E2E inclusief generated audio
- .NET backbone die “saaie” productie-eisen serieus neemt (auth, migrations, validation, telemetry)

CEO Insight

Noem niet alle tech. Noem: probleem → wat jij eigende → resultaat voor gebruiker → hoe je kwaliteit afdwong.

STAR — Ownership 0→1

- **S:** Startupfase; platform moest van PoC naar stabiele MVP voor echte gebruikers.
- **T:** Als primary engineer architectuur en levering dragen zonder het product te fragmenteren over frontend/backend/AI.
- **A:** CQRS API + Angular monorepo + AI-services; strakke client feedback loop; kwaliteitstooling (Jest/Playwright/xUnit/evals) vroeg ingebouwd.
- **R:** Productie-ready MVP binnen ~12 maanden; één ownership-lijn over de stack.
- **Les:** AI-features overleven alleen als het platform eromheen (auth, data, observatie) volwassen is.
- **Competenties:** ownership, fullstack, productdenken, tempo met kwaliteit
- **Gebruik bij:** “Vertel over jezelf”, “grootste prestatie”, “waarom jij”

STAR — Agents in productie

- **S:** Real-time conversatie/AI-workflows moesten betrouwbaar genoeg zijn voor echte sessies, niet alleen demo.
- **T:** Voice + agent orchestration leveren met duidelijke failure modes.
- **A:** LiveKit Agents + LangGraph state/checkpointing + streaming; monitoring via OpenTelemetry/App Insights; evaluatieflows met embeddings/DeepEval.
- **R:** AI-gedrag testbaar en observeerbaar; regressies eerder zichtbaar.
- **Les:** “Agent werkt” is betekenisloos zonder evals, tracing en fallback.
- **Competenties:** LLM product engineering, tool/workflow design, productie-discipline
- **Gebruik bij:** vacature-eis agents/tool use, evals/tracing

STAR — RAG met bronnen

- **S:** Antwoorden moesten steunen op bronmateriaal, niet op modelgokwerk.
- **T:** Retrieval pipeline bouwen die documenten uit blob-opslag bruikbaar maakt voor agents.
- **A:** Ingest → embeddings → ChromaDB → LlamaIndex retrieval → LLM met source-backed antwoorden.
- **R:** Traceerbare antwoorden richting gebruiker/use-case.
- **Les:** RAG-kwaliteit = data-kwaliteit + chunking + eval van retrieval, niet alleen “vector DB erbij”.
- **Gebruik bij:** energy intelligence / analyses met uitleg

STAR — Fullstack kwaliteit

- **S:** Vier Angular apps + gedeelde libs dreigden inconsistent te worden.
- **T:** Monorepo-discipline en shared libraries voor UI, auth, logging, Sentry.
- **A:** pnpm/Turbo, standalone components, NgRx + SignalR patterns, CI lint/test.
- **R:** Snellere feature-levering zonder vier keer hetzelfde wiel.
- **Gebruik bij:** fullstack, frontend-affiniteit, schaal van codebase

Echo — bewijs van AI side-project met oordeel

Eén zin: Ik bouw een Discord-agent die harm-reduction kennis veilig routeert via RAG, chat of web search — met disclaimers en privacy-scoped memory.

Waarom dit telt voor Energiemissie:

- Intent classification + tool/route keuze (agentic patroon)
- RAG op gecureerde kennis
- Product judgment: wanneer wél/niet antwoorden, tone, risk
- Eigen initiative buiten werktijd

Don't say this

Maak Echo niet groter dan Yaya. Echo is versterking; Yaya is het hoofdargument.

STAR — Intent routing

- **S:** Gebruikersvragen variëren van feitelijk tot gevoelig; één generieke chat is onveilig/onbruikbaar.
 - **T:** Elke query naar het beste pad routeren.
 - **A:** Classifier → RAG collectie / conversatie / web search; antwoorden als embeds + disclaimers.
 - **R:** Consistenter, veiliger antwoordgedrag.
 - **Les:** Agent design = policy design.
 - **Gebruik bij:** “Hoe ontwerp je agents?”, guardrails
-

Flyingdarts — bewijs van cloud en makersmentaliteit

Eén zin: Ik bouwde een real-time online dartsplatform op AWS serverless (API Gateway, Lambda, DynamoDB single-table), met een langetermijnvisie op ML dart-detectie.

STAR — Serverless product

- **S:** Real-time multiplayer + social features vroegen om schaalbare, betaalbare backend.
 - **T:** Architectuur kiezen die pieken aankan zonder zware ops.
 - **A:** Serverless API + DynamoDB single-table design; productfocus op social/real-time UX.
 - **R:** Werkend productpad + leerlijn cloud data modeling.
 - **Gebruik bij:** AWS, eigen projecten, schaaldenken
-

Donatieplatform — bewijs van schaal en verantwoordelijkheid

Eén zin: Ik werkte jaren aan een stack met 100.000+ users en meer dan een miljoen euro donaties per maand — fullstack, cloud, payments.

STAR — Geld en betrouwbaarheid

- **S:** Payment-integraties en hoge gebruikersvolumes; fouten kosten echt geld en vertrouwen.
- **T:** Features en onderhoud leveren over Angular/React/mobile/.NET met Azure/AWS.
- **A:** Integraties met Slimpay/EazyCollect/Stripe; multi-stack delivery; ook Scrum/PO verantwoordelijkheden.
- **R:** Langdurige productie-ervaring op bedrijfskritische flows.
- **Les:** “Shipt snel” betekent niets zonder respect voor money-paths en auditability — relevant voor energiefacturen/compliance-denken.

- **Gebruik bij:** schaal, .NET historie, ownership in bestaande producten
-

Hoe je projecten “verkoopt” in één adem

Template

Op [project] had ik [ownership-scope]. Het probleem was [gebruikersprobleem]. Ik koos [1–2 technische keuzes] omdat [constraint]. Resultaat: [concrete uitkomst]. Wat ik meeneem naar Energiemissie: [brug naar 250k aansluitingen / agents / .NET / evals].

Voorbeeld (60 seconden)

Op Yaya was ik primary engineer: van idee naar productie in ongeveer een jaar. We hadden real-time communicatie plus AI-agents nodig die echt live moesten. Ik zette een ASP.NET Core CQRS-API neer met Angular-frontends, en Python-services voor LangGraph-agents, voice en RAG. Belangrijker dan de stack: we dwongen evaluatie en telemetry af zodat AI-gedrag testbaar werd. Dat is precies het spiergeheugen dat je nodig hebt voor agentic features op een data platform met honderdduizenden aansluitingen.

Mini-STAR cheat (8 kaarten)

Leer deze acht “triggers” — niet uit het hoofd als toneelstuk, wel als spiergeheugen:

1. **Ownership** → Yaya 0→1
2. **Agents live** → LangGraph + LiveKit
3. **Evals/tracing** → DeepEval + App Insights
4. **RAG** → LlamaIndex/Chroma/Blob
5. **.NET diepte** → CQRS, EF migrations, SignalR
6. **Frontend op schaal** → Angular monorepo
7. **Side-project oordeel** → Echo routing/policies
8. **Bedrijfskritische schaal** → donatieplatform 100k users / payments

Als een vraag komt: kies **één** kaart, vertel 45–75 seconden, stop.

Whiteboard: Yaya in 3 minuten

Als ze vragen “schets je architectuur”:

1. Teken drie banden: **Clients (Angular)** → **.NET API** → **AI services**
2. Zet erbij: SignalR/real-time, SQL/EF, Azure blob/config/insights
3. Zoom in op AI: **LangGraph loop** met tools + streaming; voice via LiveKit
4. Zeg hardop waar kwaliteit zit: tests, evals, telemetry
5. Brug: “Bij Energiemissie zou de middelste band jullie SaaS/.NET zijn; AI-band praat via strikte tools met tenant-scope naar het data platform.”

Wat je niet tekent

- Elke NuGet-package
- Interne klassen
- Een LLM als database

STAR – Client feedback loop

- **S:** Startupfase; requirements bewegelijk; risico op bouwen wat niemand nodig heeft.
- **T:** Snel leren zonder de codebase te laten rotten.
- **A:** Korte cycles met client; PoCs bewust omzetten naar stabiele paden; kwaliteitstooling vroeg.
- **R:** MVP die in echte omgevingen gebruikt werd; minder “verrassing bij oplevering”.
- **Brug:** Energy features moeten met Product/klantpaden getest – niet alleen met synthetic prompts.

STAR – Parallel leadership (Scrum/coach)

- **S:** Naast Yaya ook faciliteren op ander product (Flutter modernisering, UI-test automation).
- **T:** Delivery helpen zonder zelf de enige builder te zijn.
- **A:** Backlog refinement met PO; test automation ownership; structure/performance verbeteren met team.
- **R:** Betrouwbaarder releases; duidelijkere stories.
- **Gebruik bij:** “Hoe werk je met Product?”, leadership zonder titel.
- **Don’t:** Dit groter maken dan je AI/.NET hoofdverhaal.

Frasering: van techniek naar Energiemissie

Yaya-detail	Brugzin
4 Angular apps	“Ik lever features tot in de UI – belangrijk als AI in de SaaS-flow moet landen.”
CQRS/MediatR	“Side-effects en queries scheiden – handig als agents writes mogen doen.”
LangGraph	“State + tools – het skelet van agentic features.”
RAG + embeddings	“Grounding – onmisbaar bij energie-advies.”
DeepEval / audio E2E	“AI test je als gedrag, niet als string-equals.”
App Insights/OTel	“Zonder traces is een agent niet te runnen in prod.”

Wat je zwijgt (bewust)

- Interne klantnamen van Methylium/Yaya tenzij publiek
- Credentials, private Azure details, ongepubliceerde metrics
- Negatieve framing van werkgever

Blijf feitelijk en professioneel; het interview is geen therapie over je huidige job.

Deel VI — AI Native cheats (interview-ready)

Alleen wat deze rol vraagt. Doel: je kunt elk begrip in **één heldere zin** uitleggen en aan Yaya/Echo koppelen.

Cheatsheet — definities in mensentaal

Term	Zeg dit
LLM	Model dat tokens voorspelt; nuttig met tools, data en policies eromheen
Prompt	Instructie + context; belangrijk, maar geen vervanging voor architectuur
Tool calling	Model vraagt een functie aan; jouw runtime valideert en voert uit
Agent	Doelgericht systeem: model + tools + state + stopcondities/policies
RAG	Eerst ophalen uit eigen data, dan genereren met bronnen
Embeddings	Numerieke betekenisvectoren om similarity/search te doen
Vector DB	Opslag/index voor embeddings (bij jou: o.a. Chroma-ervaring)
MCP	Standaardprotocol om tools/context aan agents & dev-tools te koppelen
Evals	Geautomatiseerde (en soms menselijke) tests op AI-gedrag/kwaliteit
Tracing	Spoor van prompts, tool calls, latency, errors, kosten
Fallback	Veilig alternatief als AI faalt of onzeker is
Guardrail	Harde limiet: authz, allowlists, max steps, output filters
Hallucinatie	Model verzint; bestrijd met grounding + detectie + weigeren

Agents — whiteboard in 60 seconden

User / UI

- Orchestrator (state machine / graph)
 - LLM (plan / choose tool)
 - Tool layer (RBAC, validation, timeouts)
 - Data platform (tenant-scoped)
 - LLM (observe / next step)

- Result + citations / actions
- Trace + eval hooks

Jouw bewijs: LangGraph StateGraph + checkpointing; LiveKit voice loop; Echo intent router.

Perfect answer

“De slimheid zit niet alleen in het model, maar in de tool-contracts en state machine.”

RAG — wanneer wel, hoe goed

Wel: kennis/antwoorden die in documenten of historische data staan; uitleg met bronnen.

Niet als enige: actuele meterstanden die beter via API/tool komen; writes; compliance-besluiten.

Kwaliteitshefbomen (noem er twee in een gesprek)

1. Chunking + metadata (gebouw, periode, meetserie)
2. Retrieval eval (kwam het juiste document terug?)
3. Answer eval (was het antwoord grounded?)
4. Verse indexatie / ACL's per tenant

Jouw bewijs: LlamaIndex + Chroma + OpenAI embeddings + Azure Blob ingest.

Evals — minimale productiebar

Voor elke AI-feature vóór prod:

1. **Golden set** van 20–50 echte cases (later groeien)
2. Assertions: “moet tool X aanroepen”, “mag geen tenant Y zien”, “moet bron noemen”
3. Regressie in CI op kritieke intents
4. Online monitoring: thumbs, escalation rate, cost/latency

Jouw bewijs: DeepEval/semantic checklist matching; E2E audio generatie om gedrag te valideren.

CEO Insight

Evals = hoe je belooft dat AI niet stil kapotgaat na release.

Tracing & observability

Log per request (privacy-aware):

- feature + tenant (id)
- model + token usage + cost
- tool calls + latency + success/fail
- fallback triggered?
- trace id gekoppeld aan App Insights/OTel

Jouw bewijs: OpenTelemetry / App Insights op Yaya-platform.

Fallbacks & guardrails (energy SaaS)

Risico	Guardrail
Cross-tenant leak	Authz in tool-laag, tests
Foute besparingsclaim	Bronplicht + confidente drempels
Runaway agent	Max steps, timeouts, circuit breaker
High cost	Budget caps, model routing
Write-acties	Human-in-the-loop of sterke allowlist

Don't say this

“We prompten het model om voorzichtig te zijn” als enige controle.

MCP & CLI voor het team

Waarom de vacature dit noemt: **dezelfde tools** voor mensen en agents.

Interview-pitch:

“Ik wil MCP/CLI-servers die veilige, gedocumenteerde operaties exposen — aansluitingen opvragen, aggregaties draaien, tickets maken — zodat engineers met Claude Code dezelfde contracten gebruiken als productie-agents. Dat versnelt development en verkleint drift tussen ‘wat de agent kan’ en ‘wat wij lokaal testen’.”

Eerlijkheidsregel: je hoeft geen jaren MCP-productie te claimen; wel tool-calling + developer tooling mindset.

Frameworks — hoe je erover praat

Tech	Jouw relatie	Zeg
LangGraph	Hands-on Yaya	State/tool loops gebouwd
LangChain	Gebruikt in stack	Messages/tools/ streaming
LlamaIndex	RAG path	Ingest/retrieve
Semantic Kernel / OpenAI Agents SDK	Niet claimen als expert	Ken het land- schap; kies op team+.NET-fit
Cursor / Claude Co- de / Copilot	Dagelijks	Stuur & review

Perfect answer

“Frameworks zijn replaceable. Contracts, evals en data access niet.”

.NET + AI — brugzin

Energiemissie vraagt sterke .NET. Jij hebt Python AI-services naast .NET gebouwd. Framing:

“Ik hou domain/API’s graag in .NET — saai, typed, testbaar. Model-orkestratie kan .NET of Python zijn afhankelijk van ecosystem en team. De grens trek ik bij duidelijke service contracts, niet bij hype.”

Cost & latency — één minuut

- Router model (klein) beslist; groot model alleen voor zware redenering
 - Cache retrieval en stabiele tool results
 - Prefetch data in job; LLM over samenvatting i.p.v. raw dumps
 - Stream naar UI voor perceived performance
 - Meet €/feature en p95 latency als productmetrics
-

Security one-liner

“Alles wat een agent mag, moet een normale API-user met dezelfde rol ook mogen — niet meer.”

10 flitsvragen (antwoord ≤20 seconden)

1. **Agent vs. chatbot?** → tools + state + goal vs. alleen dialoog
 2. **RAG vs. fine-tuning?** → feiten/documenten wijzigen vaak: RAG; gedrag/stijl: soms tune
 3. **Temperature?** → lager voor feitelijke/tool flows
 4. **Embeddings model wisselen?** → hermeten + herindexeren
 5. **Idempotency?** → tool retries veilig maken
 6. **Eval vs. unit test?** → gedrag/kwaliteit vs. deterministische logica
 7. **Prompt injection?** → tool-laag vertrouwt modeloutput niet blind
 8. **PII?** → redaction, dataminimalisatie, retention
 9. **Multi-agent?** → alleen bij duidelijke rol-splits; complexiteit kost
 10. **Succesmetric AI-feature?** → klantactie/besparing/tijd, niet “tokens used”
-

Uitlegpatronen (CEO → CTO)

Aan de CEO (Pieter)

Gebruik: probleem → aanpak → risico → metric.

Vermijd: framework-namen zonder vertaling.

Voorbeeld:

“We laten een agent alleen gegevens *lezen* die de gebruiker mag zien, en we meten of experts het eens zijn met de suggesties voordat we meer autonomie geven.”

Aan een CTO die bouwt

Gebruik: contracts, boundaries, failure modes, operability.

Frameworks oké, maar secundair.

Voorbeeld:

“Tool layer is de security boundary. Orchestration mag LangGraph of SK zijn; ik wil eerst idempotente, geautoriseerde capabilities en trace propagation vanuit .NET.”

Mini-ontwerp: “Besparingssignaal”-feature

Gebruik dit als concreet ontwerpgesprek (hypothese).

1. **Trigger:** schedule of user opent monitor
2. **Retrieve:** relevante meetreeksen + metadata object
3. **Analyze:** rules + LLM voor uitleg (niet voor raw math waar rules beter zijn)
4. **Tools:** GetUsage, GetBenchmarks, CreateTask (optioneel, guarded)
5. **Guardrails:** geen cross-tenant; max cost; weigeren bij lage confidence
6. **UI:** kaart in bestaande portal met bronlinks + “waarom dit signaal”
7. **Evals:** 30 golden cases; “juiste gebouw”, “geen verzonnen €”
8. **Fallback:** klassieke threshold alert zonder narratief

Perfect answer

“Ik laat het model vertellen en routeren; ik laat het niet stilletjes euro’s verzinnen.”

Begrippen die je mag durven corrigeren

Als iemand zegt “we doen AI” en bedoelt alleen een wrapper:

- Vraag naar tools, evals, authz, cost
- Bied aan hoe jij “AI-native” definieert: agents/capabilities geïntegreerd in product loops

Blijf respectvol — zij hebben het platform gebouwd; jij brengt een lens mee.

Week-1 leerplan AI×energie (zodat je concreet klinkt)

1. Glossary: aansluiting, EAN, meetverantwoordelijke, allocatie, etc. (uit interne docs/support)
2. Één tenant happy-path in de UI naspelen
3. Data access patterns: wie mag wat zien?
4. Bestaande jobs/rapportages die AI zou kunnen versnellen
5. Risicoregister: waar hallucinatie het duurst is (facturen, compliance)

Dit toont nederigheid + plan — sterker dan “ik zoom me in”.

Deel VII — Salaris, hybride en last-5-minutes

Wat de vacature al vastlegt

- **Hybride Diemen:** 1 dag thuis bij 32u, 2 dagen thuis bij 40u → dus vooral op kantoor
- Goede pensioenregeling
- Budget voor tools, AI-credits, conferenties
- Geen corporate AI-beperkingen
- Invloed op productstrategie en technische richting

Dit zijn onderhandelingshefbomen naast basis salaris: AI-credits/tools, conferentiebudget, uren (32 vs 40), groei/reviewmoment.

CEO Insight

Pieter hoort graag dat je waarde en fit eerst hard maakt. Salaris te vroeg forceren = transactional vibe.

Onderhandelingsprincipes

1. **Laat hen eerst een range noemen** als het kan (“Ik ben flexibel binnen marktconform voor deze scope — wat hebben jullie begroot?”).
2. Als je wél een getal moet geven: geef een **range**, gekoppeld aan 40u + hybride + scope (AI-native ownership).
3. Onderhandel **total package**: base, pensioen, vakantiedagen, hardware, AI-budget, opleidings/conferentie, review na 6 maanden.
4. Wees bereid 32u vs 40u expliciet te maken — dat verandert thuisdagen én inkomen.

Hoe je het zegt (toon)

“Ik zoek een marktconform pakket voor een AI-native fullstack met ownership over de intelligence layer. Belangrijker nu: of we elkaar scherp zien op scope en 90-dagen impact. Als de fit klopt, komen we eruit op de cijfers.”

Don'ts

Don't say this

- Exact bodemsalaris als openingsbod
- “Remote fulltime is een must” (botst met hybride tekst)
- Vergelijken met FAANG-cijfers zonder NL SaaS-context
- Package details vóóordat ze enthousiast zijn over jou

Marktkaders (oriënteerend, geen garantie)

Voor een senior/lead-achtige fullstack met AI-productie-ervaring in NL SaaS liggen gesprekken vaak in een brede senior-band; **vul jouw concrete range in vóór het gesprek** op basis van recruiter-input en recente offers. Schrijf hem hieronder:

- Jouw range (40u): € _____ – € _____
- Bodem (walk-away): € _____
- Nice-to-haves: _____

(Vul dit in met de recruiter in het oefengesprek — laat het niet leeg tot vrijdagochtend.)

Vragen die wél mogen over voorwaarden

- “Hoe zien review- en groeipaden eruit in een klein engineering team?”
 - “Hoe concreet is het budget voor AI-credits en tools in de praktijk?”
 - “Hoe ziet een typische week eruit qua kantoor/Diemen voor dit team?”
 - “Hoe wordt succes van deze rol gemeten na zes maanden?”
-

Last 5 minutes — cheat sheet

Print of open dit vlak voor je binnenloopt.

Energiemissie in 20 seconden

SaaS voor energiebeheer (monitor, aansluitingen, factuurcontrole) voor overheid/zorg/retail; AI-native rebuild; agentic features op 250k+ aansluitingen — geen chatbot.

Jij in 20 seconden

Product-minded fullstack; .NET + Angular + productie-agents; Yaya 0→1 in ~12 maanden; wil intelligence layer hier shippen.

Cultuur — 3 woorden

Autonomie · AI-first · Innovatie

Plus: **jij stuurt, AI bouwt.**

90 dagen — 3 bullets

1. Domein + data contracts snappen
2. Eerste AI-feature live met evals/fallback
3. Standaard + MCP/CLI voor herhaalbaar AI-bouwen

3 bewijzen

1. **Ownership:** Yaya primary engineer
2. **AI prod:** LangGraph/LiveKit/RAG + DeepEval/tracing
3. **.NET:** ASP.NET Core CQRS + Azure

Top don'ts

- Chatbot als eerste idee

- MCP = “ken ik niet” zonder brug
- Alleen Copilot-autocomplete als AI-verhaal
- Energiedomein bluffen

3 vragen aan hen

1. Succes van AI-rebuild over 6 maanden?
2. Grootste frictie data platform ↔ features?
3. Hoe kiezen jullie betaalde AI-features?

Ademhaling

Schouders laag. Antwoord 60–90 seconden. Stop. Laat stilte bestaan.

Avond-vóór checklist

- ☐ Range + bodem ingevuld met recruiter-input
- ☐ Yaya STAR ownership + agents één keer hardop
- ☐ Vacature nog eens gescand (90 dagen + cultuur)
- ☐ Pieter = SaaS CEO (ex-Shypple COO), niet “alleen hiring manager”
- ☐ Route/tijd Diemen + hybride verwachting scherp
- ☐ Deze last-5 pagina nog één keer gelezen

Ochtend-van (10 minuten)

1. Lees last-5
2. Zeg 30-seconden pitch hardop
3. Één glas water
4. Telefoon op stil

Je hebt dit gesprek al voorbereid. Nu alleen nog laten horen.

Onderhandelingsscripts (kort)

Als zij om een getal vragen

“Voor deze scope — AI-native fullstack met ownership over de intelligence layer, hybride Diemen, 40 uur — zit ik op **[range]**. Ik kijk naar het totale pakket: pensioen, tools/AI-credits, groei. Als de rol en 90-dagen impact kloppen, ben ik constructief.”

Als het bod onder je bodem ligt

“Dank — fijn dat we hier open over praten. Op **[bod]** wordt het voor mij krap gegeven de scope. Kan er beweging in base, of in een review na zes maanden gekoppeld aan AI-features in productie? AI-budget en conferenties helpen, maar base blijft leidend.”

Als ze 32u voorstellen

“32u kan interessant zijn voor focus/duurzaam tempo. Dan wil ik scherp: scope van de rol, thuisdag-regeling, en of ownership van de AI-laag realistisch blijft. Ik wil geen ‘alles van 40u in 32u’ stilzwijgend.”

Presence (praktisch, geen zweverig advies)

- Zit iets naar voren bij antwoorden die je wilt laten landen
 - Bij tech-diepgang: handen mogen tekenen in de lucht / op papier
 - Bij CEO-value vragen: langzamer praten, kortere zinnen
 - Niet excuses voor mijn imperfecte Nederlands/Vlaams — je bent vloeiend; wees gewoon helder
 - Eindig antwoorden met een punt, niet met “haha weet ik niet of dit klopt”
-

Na het gesprek (dezelfde dag)

1. Noteer vragen die je niet zag aankomen
2. Noteer waar je wafelde — herschrijf 5 regels in dit boek
3. Bedankmail: kort, specifiek (“vond het gesprek over X scherp; blijf enthousiast over Y”)
4. Geen novel; geen salaris heronderhandelen in de thank-you tenzij gevraagd